```
--- djangoapi\core\myLib\pgOperations.py ---

# -*- coding: utf-8 -*-
'''
#pgOperations module
A simple and light weight module to perform operations in PostgreSQL and PostGIS.

This module facilitates to perform the most common operations:
insert, delete, update, select, create and delete databases.

The class methods receive Python dictionaries, create the SQL sentences and perform the
operations.
The methods are able to work also with PostGIS geometries, using WKT representations.

This library depends on the
<a href="https://www.psycopg.org/" target="_blank">psycopg v3</a> library.

This library requires:
    - Python >= 3.
    - Psycopg2 >= 3
'''

from typing import Union, Any

import psycopg, os
from psycopg import sql

# import json

# psycopg2.extensions.register_type(psycopg2.extensions.UNICODE)
# psycopg2.extensions.register_type(psycopg2.extensions.UNICODEARRAY)

class PgConnection():
    """Class to store a Psycopg2 connection and cursor, which are the objects
    used to perform the transactions in the database.

     This class do not do anything interesting, but an instance of this class, or an
instance of
    [PgConnect][src.pgOperations.pgOperations.PgConnect], are necessary to initialize
    the class [PgOperations][src.pgOperations.pgOperations.PgOperations], which have the

    real utilities of this library.

                This    class    is    the    base    class    for    the    class
[PgConnect][src.pgOperations.pgOperations.PgConnect],
    which allows creating the Psycopg2 connection with the user credentials.
    """
    conn=None
    """psycopg2 connection object - class variable"""
    cursor=None
    """psycopg2 cursor object - class variable"""
    def __init__(self, psycopg3Connection):
        """
```

```
        Constructor

        Examples:
            >>> import psycopg2
            >>> from pgOperations import pgOperations as pg
            >>> conn=psycopg2.connect(database="pgoperationstest", user="postgres",
                    password="postgres", host="localhost", port=5432)
            >>> pgConnection = pg.PgConnection(conn)
            >>> pgOperations = pg.PgOperations(pgConnection)

        args:
            psycopg2Connection (psycopg2 connection): a psycopg2 connection
        """
        self.conn = psycopg3Connection
        self.cursor=self.conn.cursor()

    def disconnect(self):
        """Closes the cursor and the connection.

        It is important to close the connections because the number
        of connections alive are limited.
        In PostgreSQL, this number is configurable, and also there is
         a garbage collector to close unused connections, but the recommendation
        is to close the connection once you have finished to release resources.

          The recommendation is use the connection to perform all the transactions you
need,
        commit the changes and close the Psycopg2 cursor and the connection.
        """
        self.cursor.close()
        self.conn.close()

    def commit(self):
        """Commits the changes in the database. The changes will not be performed until
           you have committed the transaction. You will not see the changes until you
commit.
        """
        self.conn.commit()

class PgConnect(PgConnection):
    """Class to create and store a Psycopg2 connection and cursor, which are the objects
    used to perform the transactions in the database.

    An instance of this class, or and instance
    of the class [PgConnection][src.pgOperations.pgOperations.PgConnection],
    is necessary to initialize the class
    [PgOperations][src.pgOperations.pgOperations.PgOperations], which have the
    real utilities of this library.
    """
    database:str=None
    user:str=None
    password:str=None
    host:str=None
```

```
    port:Union[str,int]=None

        def __init__(self, database: str,user: str,password: str,host: str ,port:
Union[str,int]):
        """Constructor

        Examples:
            >>> from pgOperations import pgOperations as pg
            >>> pgc=pg.PgConnect(database='pgoperationstest', user="postgres",
            >>>     password="postgres", host="localhost",port=5432)
            >>> pgo=pg.PgOperations(pgConnection=pgc)
            >>> pgc.disconnect()

        Args:
            database: The database name to connect.
            user: The user to connect.
            password: The user password.
            host: Host address.
            port: The port number where PostgreSQL is listening to.
        """
               self.conn=psycopg.connect(dbname=database, user=user, password=password,
host=host, port=port)
        PgConnection.__init__(self,self.conn)
        self.database=database
        self.user=user
        self.password=password
        self.host=host
        self.port=port
        #self.conn.autocommit = True  #Todos los execute se envían

class PgDatabases():
    """
    Class to create and delete databases. To use this class it is necessary to have
    an instance of the class PgConnect.
    """
    pgConnect: PgConnect = None

    def __init__(self, pgConnect: PgConnect):
        """Constructor

        Examples:

                            >>>  pgc=pg.PgConnect(database='postgres',  user="postgres",
password="postgres",
                        host="localhost",port=5432)
            >>> pgdb=pg.PgDatabases(pgConnect=pgc)
            >>> pgc2=pgdb.createDatabase(databaseName="pgoperationstest",
                        addPostgisExtension=True,closeNewConection=False)
            >>> pgc2.cursor.execute("create schema d")
                >>> pgc2.cursor.execute("create table d.points (gid serial primary key,
description varchar, depth double precision, geom geometry('POINT',25831))")
            >>> pgc2.commit()
            >>> pgc2.disconnect()
```

```
            >>> pgc.disconnect()

        Args:
            pgConnect: PgConnect instance.
        """
        self.pgConnect=pgConnect
        self.pgConnect.conn.autocommit=True #necesario para que cree y borre bbdd

    def databaseExists(self, databaseName:str)->bool:
                cons=f"SELECT  EXISTS  (SELECT  1  FROM  pg_database  WHERE  datname  =
'{databaseName}')";
        self.pgConnect.cursor.execute(cons)
        r=self.pgConnect.cursor.fetchall()
        #print('database exists',r)
        return r[0][0] #ya es True o False

    def createDatabase(self, databaseName: str, addPostgisExtension: bool = True,
            closeNewConnection:bool = False) -> PgConnect:
        """Create a database.

        Examples:

            >>> pgc2=pgdb.createDatabase(databaseName="pgoperationstest",
                    addPostgisExtension=True,closeNewConection=False)
        Args:
            databaseName: The database name.
               addPostgisExtension: Whether or not add the PostGIS extension to the new
database. Default True.
                closeNewConnection: Whether or not to close the connection to the new
database. Default True.

        Returns:
            A PgConnect instance to manage the newly created database.
        """
        self.pgConnect.cursor.execute('create database ' + databaseName)
        pgConnect: PgConnect=PgConnect(database=databaseName, user=self.pgConnect.user,
            password=self.pgConnect.password, host= self.pgConnect.host,
            port=self.pgConnect.port)
        if addPostgisExtension:
            pgConnect.cursor.execute("create extension postgis")
            pgConnect.conn.commit()
        if closeNewConnection:
            pgConnect.disconnect()
        return pgConnect

    def dropDatabase(self,databaseName: str):
        """
        Deletes a database.

        Examples:

            >>> pgdb.dropDatabase("pgoperationstest")
        """
```

```
        self.pgConnect.cursor.execute('drop database ' + databaseName)

class FieldsAndValuesBase():
    """Class to store the information that
    [pgInsert][src.pgOperations.pgOperations.PgOperations.pgInsert]
    and [pgUpdate][src.pgOperations.pgOperations.PgOperations.pgUpdate] need
    to work.

                          Is      the      base      class      for      the      class
[FieldsAndValues][src.pgOperations.pgOperations.FieldsAndValues]

    """
    str_field_names: str = None
    list_field_values: str = None
    str_s_values: str = None
    def __init__(self,str_field_names: str,list_field_values: str, str_s_values: str):
        """Constructor

        Examples:
            >>>fieldsAndValuesBase=FieldsAndValuesBase(
            >>>     str_field_names="depth, description, geom",
            >>>     list_field_values=[12.15, "water well","POINT(100 200)"],
                                                                  >>>
str_s_values="%s,%s,st_transform(st_geometryfromtext(%s,25830),25831)"
            >>>)

        Args:
            str_field_names: String with the name of the fields of a table.
            list_field_values: List with the values of the fields.
                                Exactly the same number and in the same order.
            str_s_values: String with a %s for each field value, to use in the
                                execute method of a psycopg2 cursor.
        """
        self.str_field_names=str_field_names
        self.list_field_values=list_field_values
        self.str_s_values=str_s_values


class GeometryFieldOptions():
     """Class with the details of the geometry field in the dictionary `d`, argument of
the
    methods [pgInsert][src.pgOperations.pgOperations.PgOperations.pgInsert] and
    [pgUpdate][src.pgOperations.pgOperations.PgOperations.pgUpdate].

      With this class you can set the geometry field name, the current SRC of the
geometry,
    and the new SRC, in case you want ro reproject it. The SRC must be a
    <a href="https://epsg.io/" >EPSG</a> code.
    """
    geom_field_name=None,
    epsg=None
    epsg_to_reproject=None
    def __init__(self,epsg: Union[str, int], geom_field_name: str = 'geom',
```

```
         epsg_to_reproject: Union[str, int]=None):
        """Constructor

        Examples:

            >>>geometryFieldOptions=pg.GeometryFieldOptions(geom_field_name="geom",
                    epsg='25830',epsg_to_reproject="25831")

        Args:
            epsg: Current EPSG SRC code of the geometry.
            geom_field_name: Table geometry field name.
            epsg_to_reproject: On inserting or getting the geometry, the
                geometry will be reprojected to this new SRC.
        """
        self.geom_field_name=geom_field_name
        self.epsg=str(epsg)
        if epsg_to_reproject is not None:
            self.epsg_to_reproject=str(epsg_to_reproject)

class SelectGeometryFormat():
    """
    Contains the allowed formats to get the coordinates of a geometry.
    The possible values are 'text', 'geojson', or 'binary'.

    This class is useful to create an instance of the class SelectGeometryFieldOptions.
    Examples:
        >>>gf=pg.SelectGeometryFormat()

>>>gfo=pg.SelectGeometryFieldOptions(geom_field_name='geom',select_geometry_format=gf.ge
ojson, epsg_to_reproject='25831')

>>>fieldNames=pgo.pgGetTableFieldNames('d.points',gfo,list_fields_to_remove=['descriptio
n'],returnAsString=True)
        >>>wc=pg.WhereClause(where_clause='gid=%s',where_values_list=[3])
                                                >>>res=pgo.pgSelect(table_name='d.points',
string_fields_to_select=fieldNames,whereClause=wc)
    """
    text: str = 'text'
    geojson: str ='geojson'
    binary: str = 'binary'

class SelectGeometryFieldOptions():
    """
    This class allows to specify the format to retrieve the geometry field name, e.g.
                              'geom',       'st_astext(geom)',       'st_asgeojson(geom)',
'st_transform(st_asgeojson(geom),EPSG),
    or 'st_transform(st_astext(geom),EPSG).

    This is useful to get the field names in a proper string to select rows.

    Examples:
        >>>gf=pg.SelectGeometryFormat()
        >>>gfo=pg.SelectGeometryFieldOptions(geom_field_name='geom',
```

```
            select_geometry_format=gf.geojson, epsg_to_reproject='25831')#

>>>fieldNames=pgo.pgGetTableFieldNames('d.points',gfo,list_fields_to_remove=['descriptio
n'],
            returnAsString=True)
        >>>wc=pg.WhereClause(where_clause='gid=%s',where_values_list=[3])
                                                >>>res=pgo.pgSelect(table_name='d.points',
string_fields_to_select=fieldNames,whereClause=wc)
    """
    geom_field_name: str = None
    select_geometry_format: str = None
    epsg_to_reproject: Union[str,int] = None

    def __init__(self, geom_field_name:str='geom',
            epsg_to_reproject: Union[str, int]=None,
            select_geometry_format: str = 'text'):
        """Constructor

            geom_field_name: The geometry field name.
            epsg_to_reproject: The EPSG code to reproject the geometry
            select_geometry_format: The format to retrieve the geometry.
                The value can be: 'text', 'geojson', or 'binary'. Otherwise
                raises an error.
        """
        if select_geometry_format not in ['binary', 'geojson','text']:
                        raise  Exception("Select  geometry  format  not  in  ['binary',
'geojson','text']")
        self.geom_field_name=geom_field_name
        self.epsg_to_reproject=epsg_to_reproject
        self.select_geometry_format=select_geometry_format

class FieldsAndValues(FieldsAndValuesBase):
    """Create a SQL expression from a Python dictionary.

    This class is used in the methods
    [pgInsert][src.pgOperations.pgOperations.PgOperations.pgInsert] and
    [pgUpdate][src.pgOperations.pgOperations.PgOperations.pgUpdate] of the class
    [PgOperations][src.pgOperations.pgOperations.PgOperations].

    """
    d:dict = None
    list_fields_to_remove: list=None

    def __init__(self, d:dict, list_fields_to_remove: list=None,
        geometryFieldOptions: GeometryFieldOptions =None):
        """Constructor

        Args:
            d: Dictionary key-value, where the keys are the names of the table fields,
                and the values the values of the fields,
                  e.g. {"depth":12.15, "description":"water well", "geom":"LINESTRING(100
200, 200 200)"}.
                It is not necessary to have a geometry field in the dictionary.
```

```
                If there is a geometry field, the value must be in
                                                                                <a
href="https://en.wikipedia.org/wiki/Well-known_text_representation_of_geometry"
                    target="_blank">WKT</a> format. The other details of the geometry,
                    as the SRC of the geometry, must be specified in
                    an instance of the

[GeometryFieldOptions][src.pgOperations.pgOperations.GeometryFieldOptions] class,
                    in the parameter geometryFieldOptions of this contructor.
            list_fields_to_remove: list with the dictionary keys to exclude of the SQL
expression.
                The corresponding field for the row in the table will not be affected by
the insertion,
                or update.
                For example ['gid'] will remove the gid from the expressions and
                  list of values. The value for the field in an insert will be null. The
value
                for the field in an update will not be changed.
                Set to None if you do not want to remove any field.
                  If the field to remove is not in the dictionary `d` any error will be
raised.
            geometryFieldOptions: If there is a geometry field in the table,
                                                                                a
[GeometryFieldOptions][src.pgOperations.pgOperations.GeometryFieldOptions]
                    class instance must be created, to give the details of the geometry:

                    the field name, the SRC,
                    and the new SRC, in case of a reprojection be required.
        """
        self.__dict_to_string_fields_and_vector_values(
            d, list_fields_to_remove, geometryFieldOptions)

    def __dict_to_string_fields_and_vector_values(
            self,
            d: dict,
            list_fields_to_remove: list,
            geometryFieldOptions: GeometryFieldOptions):

        if geometryFieldOptions is not None:
            geom_field_name=geometryFieldOptions.geom_field_name
            epsg=geometryFieldOptions.epsg
            epsg_to_reproject=geometryFieldOptions.epsg_to_reproject

        #remove the fields to delete
        if list_fields_to_remove != None:
            for i in range(len(list_fields_to_remove)):
                key=list_fields_to_remove[i]
                if key in d:
                    del d[key]

        #forms the tree values returned in the dictionary
        it=list(d.items())
        str_name_fields=""
```

```
        list_values =[]
        str_s_values=""
        for i in range(len(it)):
            str_name_fields = str_name_fields + it[i][0] + ","
            #change the '' values by None
            if it[i][1]=="":
                list_values.append(None)
            else:
                list_values.append(it[i][1])

            if geometryFieldOptions is None:
                str_s_values=str_s_values + "%s,"
            else:
                if it[i][0] != geom_field_name:
                    str_s_values=str_s_values + "%s,"
                else:
                    if epsg_to_reproject is None:
                        st="st_geometryfromtext(%s,{epsg}),".format(epsg=epsg)
                        str_s_values=str_s_values + st
                    else:

st="st_transform(st_geometryfromtext(%s,{epsg}),{epsg_to_reproject}),".format(epsg=epsg,
epsg_to_reproject=epsg_to_reproject)
                        str_s_values=str_s_values + st
                    #(%s,st_geometryfromtext(%s,25830))

        str_name_fields=str_name_fields[:-1]
        str_s_values=str_s_values[:-1]

        #Initialize  the base class properties
        FieldsAndValuesBase.__init__(self,str_name_fields, list_values, str_s_values)

class WhereClause():
    """
    Class to store the where clause data of the SQL query.
    The where clause must not be written with values, e.g.
    `'depth > 25.4 and owner_name = "Juan Andrés"'` <-- **do not do that**.

    Instead, the where clause must be divided in two parts:

    * A string expression with this format: `'depth > %s and owner_name = %s'`.
            This expression is stored in the `where_clause` property.
    * The values to replace the `%s` by real values, e.g. `[25.4,"Juan Andrés"]`.

    The `execute` method of the `cursor` object of the `Psycopg2` library will
    put the values into the where string expression, in the correct way. You do
    not have to worry about quotes in the values. See the section <i>Passing parameters
    to SQL queries</i> in the <a href='https://www.psycopg.org/docs/usage.html'
    target='_blank'>Psycopg2</a> documentation.

    """
    where_clause:str=None
    where_values_list: list = None
```

```
    def __init__(self, where_clause:str,where_values_list: list) -> None:
        """Constructor.

        Args:
            where_clause: The where condition expression, to select the rows to
                update, or delete. Eg: `'description = %s and depth = %s'`.
                The `where` word must not be in the expression. The values for the where
                condition must not be specified in the string. Instead %s must be used.
                The values of the where condition must be specified in the property
                `where_values_list`. **Do not quote the %s in the where string**.
            where_values_list: List of the values of the %s in the
                property `where_clause`, in the order of the %s. e.g. if the
                where condition is e.g. `'where description = %s and depth = %s'`
                the list of values must have two values, first one the value
                for the field `condition` and the second one the value for
                the field `depth`, e.g. `["This is the description", 25.3]`.
        """
        self.where_clause = where_clause
        self.where_values_list=where_values_list
    def printProperties(self):
        print('where_clause: ', self.where_clause)
        print('where_values_list', self.where_values_list)

class PgOperations():
    """
    Perform the most common operations with PostGIS: insert, delete, update and select.
    To initialise this class it is necessary a
    [PgConnection][src.pgOperations.pgOperations.PgConnection],
    or a [PgConnect][src.pgOperations.pgOperations.PgConnect] instance.

    All the next examples use the variable `pgc`, the database,
    and the table `d.points`, created in the The
    <a href="../tutorials/#create-a-table" >Create table</a>
    section. The `d.points` table has the fields:
     `gid`, `description`, `deph` and `geom`.
    """

    query: str = None
    global_print_queries: bool = None
    pgConnection : Union[PgConnect, PgConnection]=None
    autoCommit: bool = None

    def __init__(self, pgConnection: Union[PgConnect, PgConnection],
                 autoCommit: bool = True, global_print_queries: bool = False):
        """Constructor

        Examples:

                                >>>   pgo=PgOperations(pgConnection=pgc,   autoCommit=True,
global_print_queries=True)

        Args:
            pgConnection: PgConnection or PgConnect instance.
```

```
            autoCommit: performs a commit after every db operation.
            global_print_queries: For debugging purposes. If True
                all the methods will print the
                      SQL sentences. All the methods of this class have the parameter
`print_query`,
                if any of both variables, `global_print_queries` or `print_query`,
                is True, the SQL queries and data will be printed.
        """
        self.pgConnection=pgConnection
        self.autoCommit=autoCommit
        self.global_print_queries=global_print_queries

    def pgInsert(self, table_name: str,
            fieldsAndValues: Union[FieldsAndValuesBase,FieldsAndValues],
            str_fields_returning: str = None, print_query: bool=False)->list:
        """Inserts a row in a table.

        See examples of use in  <a href="../tutorials/#insert" >Examples of insert</a>.

        Args:
            table_name: Table name to insert the row.
            fieldsAndValues: Object containing SQL expression pars of the SQL, and the
                list of values.
              str_fields_returning: A string with the field values, comma separated, of
the
                inserted row to be returned, e.g. ´'gid,date'´. The values are put in a
                    dictionary, and the dictionary is put in a list, e.g. `[{'gid': 25,
'date': '2022-11-13'}]`.
                If this parameter is not set None is returned.
               print_query: If `True` the SQL sentence is printed in the screen. This
option
                may be is interesting for debugging.

        Returns:
              An empty list or a list with a dictionary, depending of the value of the
parameter ´str_fields_returning´
        """

        conn=self.pgConnection.conn
        cursor=self.pgConnection.cursor

        str_field_names=fieldsAndValues.str_field_names
        list_field_values=fieldsAndValues.list_field_values
        str_s_values=fieldsAndValues.str_s_values

                                    #cons_ins='insert    into    {0}    ({1})    values
(%s,st_geometryfromtext(%s,25830))'.format(table_name, string_fields_to_set)
                  cons_ins='insert  into  {0}  ({1})  values  ({2})'.format(table_name,
str_field_names, str_s_values)

        if str_fields_returning != None:
            cons_ins =cons_ins + ' returning ' + str_fields_returning
```

```
        if self.global_print_queries or print_query:
            print('pgInsert')
            print('Query: ', cons_ins)
            print('Values: ', list_field_values)
        self.query=cons_ins
        cursor.execute(cons_ins,list_field_values)

        if self.autoCommit:
            conn.commit()

        if str_fields_returning != None:
            returning=cursor.fetchall()
            fieldNames=str_fields_returning.split(",")
            d={}
            i=0
            for field in fieldNames:
                d[field.strip()]=returning[0][i]
                i=i+1
            return [d]
        else:
            return []

    def pgUpdate(self, table_name: str, fieldsAndValues: FieldsAndValues,
                 whereClause: WhereClause=None,
                 print_query: bool=False) -> int:
        """
        Updates a table

        See examples of use in  <a href="../tutorials/#update" >Examples of update</a>.

        Args:

            table_name: table name included the schema. Ej. "d.linde".
                Mandatory specify the schema name, even if the table
                is in the schema `public`: "public.tablename".
            fieldsAndValues: Object containing SQL expression pars of the SQL, and the
                list of values.
              whereClause: Data of the where clause. If it is none all the rows will be
updated.
                print_query: print_query: If `True` the SQL sentence is printed in the
screen. This option
                may be is interesting for debugging.

        Returns:
            The number of updated rows.
        """
        conn=self.pgConnection.conn
        cursor=self.pgConnection.cursor

        str_field_names=fieldsAndValues.str_field_names
        list_field_values=fieldsAndValues.list_field_values
        str_s_values=fieldsAndValues.str_s_values
```

```
        cons='update {table_name} set ({str_field_names}) = row({str_s_values})'.format(

table_name=table_name,str_field_names=str_field_names,str_s_values=str_s_values)
        if whereClause != None:
            cons += ' where ' + whereClause.where_clause
            cursor.execute(cons,list_field_values + whereClause.where_values_list)
        if self.global_print_queries or print_query:
            print('pgUpdate')
            print ('Query: ' + cons)
            if whereClause != None:
                whereClause.printProperties()
            print('New field values: ', list_field_values)
            print('Number of rows updated: ', cursor.rowcount)
        if self.autoCommit:
            conn.commit()

        self.query=cons
        return cursor.rowcount

     def pgDelete(self, table_name: str, whereClause: WhereClause = None, print_query:
bool=False) -> int:
        """
        Delete rows from a table. See an example of use in
        <a href="../tutorials/#delete" >Example of delete</a>.

        Args:

            table_name: table name included the schema, e.g. `d.linde`.
                It is mandatory to specify the schema name: `public.tablename`
            whereClause: Data of the where clause. If it is None, all the rows
                will be deleted.
            print_query: If True, will print the SQL query, form debugging purposes.

        Returns:
            The number of deleted rows.
        """
        conn=self.pgConnection.conn
        cursor=self.pgConnection.cursor
        cons='delete from {table_name}'.format(table_name=table_name)
        if whereClause != None:
            cons += ' where ' + whereClause.where_clause
            cursor.execute(cons, whereClause.where_values_list)
        else:
            cursor.execute(cons)
        conn.commit()
        self.query=cons
        if self.global_print_queries or print_query:
            print('pgDelete')
            print('Query: ', cons)
            if whereClause != None:
                whereClause.printProperties()
            print('Number of rows deleted: ', cursor.rowcount)
```

```
        return cursor.rowcount

    def pgDeleteWithFiles(self, table_name: str, field_name_with_file_name: str,
            whereClause: WhereClause=None, base_path: str=None,
            print_query: bool=False)->dict:
        """
        **This method only has been tested in Linux systems**

        Deletes the selected rows, and their associated files in the hard disk.
        The rows have to have a field with the file names to delete. The file names
        can contain an absolute path, or a relative path. It is possible to complete
        relative paths with the parameter `base_path`.

        Before the rows be deleted, the files are deleted.

        See an example of use in
        <a href="../tutorials/#delete-rows-and-files" >Delete rows and files</a>.

        Args:

            table_name: Table name with schema, e.g. `public.customers`.
             field_name_with_file_name: field in the table which contains the file name,
e.g. `img`.
            whereClause: Data of the where clause. If it is None, all the rows and files
                            will be deleted.
              base_path: Base path to prepose to the file names. Can end with or without
`/`, e.g.
                 `/home/user/app/media/customers/img`.
              print_query: for debugging purposes. If true will print the SQL sentences
and values.

        Returns:
              Dictionary with the following information. Number of rows deleted, list of
file names deleted, list of filenames not deleted, the base path.

        """
        r=self.pgSelect(table_name=table_name,
            string_fields_to_select=field_name_with_file_name,
            whereClause= whereClause, print_query=print_query)
        deletedFileNames=[]
        notExistingFilenames=[]
        for row in r:
            deleted=self.pgDeleteFileInRow(row, field_name_with_file_name,base_path)
            if deleted:
                deletedFileNames.append(row[field_name_with_file_name])
            else:
                notExistingFilenames.append(row[field_name_with_file_name])

        n=self.pgDelete(table_name, whereClause=whereClause)
        return {'numOfRowsDeleted': n, 'deletedFileNames': deletedFileNames,
            'notDeletedFilenames': notExistingFilenames, 'base_path': base_path}

       def  pgDeleteFileInRow(self,row:  dict,  field_name_with_file_name:str,  base_path:
```

```
str=None) -> bool:
        """
        If you have a row stored in a dictionary, this function
        deletes a file in the file system. The the filename must be one of the values
        of the dictionary. **This method has only been texted in Linux systems**.

        Examples:

            >>>d={'gid': 25, 'description': 'customer', 'img': '00012325.jpg'}
            >>>resp=pgo.deleteFileInRow(row=d,field_name_with_file_name='img',
                base_path='/home/user/media/customers/img')

        Args:

            row: A Python dictionary with the filename to delete,
                e.g. {'gid': 25, 'description': 'customer', 'img': '00012325.jpg'}
            field_name_with_file_name: field in the table which contains the file name.
            base_path: If the `field_name_with_file_name` field contains relative paths,
                    or only the file name, you can specify in this parameter the
                    base path to complete the absolute path to the file. It does not
                    matter if the base_path ends in the character `/` or not,
                    e.g. `/home/user/media/images/`, or  `/home/user/media/images`.


        Returns:
            True if the file could be deleted. Otherwise False.
        """

        if base_path is not None:
            #print(base_path[len(base_path)-1])
            if base_path[len(base_path)-1]== '/':
                base_path = base_path
            else:
                base_path = base_path + '/'

        imageName=row[field_name_with_file_name]
        if base_path is not None:
            imageName = base_path + imageName

        if os.path.isfile(imageName):
            os.remove(imageName)
            return True
        return False

    def pgSelect(self, table_name: str, string_fields_to_select: str = "*",
                whereClause: WhereClause = None,
                get_rows_as_dicts: bool=True,
                limit: int=100, orderBy: str=None, groupBy: str=None,
                print_query: bool=False)->list:
        """
        Select rows of a table.

        See examples of use in  <a href="../tutorials/#select" >Examples of select</a>.
```

```
        Args:
            table_name: table name included the schema, e.g. "d.linde".
                Mandatory specify the schema name, even if the table is
                in the schema `public`: "public.tablename".
            string_fields_to_select: string with the fields to select, comma separated,
                e.g. 'gid, description, area, st_asgeojson(geom)'. You can use '*'
                to select all table fields.
              whereClause: Data of the where clause. If it is none all the rows will be
selected.
            get_rows_as_dicts: If True the function will return a list of dictionaries,
                each dictionary representing a selected row. If False the function
                  will return a list of tuples. Each list element represents a selected
row.
            limit: The maximum rows to return.
            orderBy: Order by SQL clause, e.g. "depth desc". Will order the results
                     by the field depth descending. **The words `order by` must not be
specified**.
            groupBy: Group by SQL clause, e.g. "depth". Will group the results
                by the field depth. **The words `group by` must not be specified**.
                print_query: print_query: If `True` the SQL sentence is printed in the
screen. This option
                may be is interesting for debugging.

        Returns:
            An empty list, if there is not any row selected.
               A list of dictionaries, each dictionary representing a selected row, if
`get_rows_as_dicts` is True.
                     A list of lists, each list representing a selected row, if
`get_rows_as_dicts` is True.
        """
        cursor=self.pgConnection.cursor

        if orderBy== None:
            orderBy=""
        else:
            orderBy = "order by " + orderBy

        if groupBy== None:
            groupBy=""
        else:
            groupBy = "group by " + groupBy

          #executes the string. The list_val_cond_where has the values of the %s in the
select string by order
        if whereClause == None:
            if get_rows_as_dicts:
                        cons='SELECT array_to_json(array_agg(registros)) FROM (select
{string_fields_to_select} from {table_name} {groupBy} {orderBy} limit {limit}) as
registros'.format(
                    string_fields_to_select=string_fields_to_select,
                     table_name=table_name,limit=limit, orderBy=orderBy, groupBy=groupBy)
```

```
            else:
                    cons = 'select {string_fields_to_select} from {table_name} {groupBy}
{orderBy} limit {limit}'.format(
                     string_fields_to_select=string_fields_to_select,
                     table_name=table_name,limit=limit, orderBy=orderBy, groupBy=groupBy)
            self.query=cons
            cursor.execute(cons)
        else:
            if get_rows_as_dicts:
                        cons='SELECT  array_to_json(array_agg(registros))  FROM  (select
{string_fields_to_select}  from  {table_name}  {cond_where}  {groupBy}  {orderBy}  limit
{limit}) as registros'.format(

string_fields_to_select=string_fields_to_select,table_name=table_name,
                     cond_where= " where " + whereClause.where_clause, limit=limit,
                     orderBy=orderBy, groupBy=groupBy)
            else:
                   cons='select {string_fields_to_select} from {table_name} {cond_where}
{groupBy} {orderBy} limit {limit}'.format(

string_fields_to_select=string_fields_to_select,table_name=table_name,
                     cond_where= " where " + whereClause.where_clause, limit=limit,
                     orderBy=orderBy, groupBy=groupBy)
            self.query=cons
            cursor.execute(cons, whereClause.where_values_list)

        #gets all rows
        lista = cursor.fetchall()
        if get_rows_as_dicts:
            r=lista[0][0]
        else:
            r=lista
        if r == None:
            r = [] #there wheren't selected rows
        else:
            #in ubuntu 14.04 r is a string, in 16.04 is a list
            #if it is a string is converted to list
            if type(r) is str:
                r=json.loads(r)

        if self.global_print_queries or print_query:
            print('pgSelect')
            print("Query: ", cons)
            if whereClause != None:
                whereClause.printProperties()
            print("Num of selected rows: ", len(r))

        return r

    def pgGetTableFieldNames(self, table_name: str,
            selectGeometryFieldOptions: SelectGeometryFieldOptions=None,
            list_fields_to_remove: list=None,
            returnAsString=False, print_query: bool = False)-> Union[list,str]:
```

```
        """
        Returns the field names of a table. Depending of the `returnAsString` parameter
        returns a string or a list.
        The geometry field name can be returned with one of the following formats:
                                    'geom',    'st_astext(geom)',    'st_asgeojson(geom)',
'st_transform(st_asgeojson(geom),EPSG),
        or 'st_transform(st_astext(geom),EPSG). The objetive with this is the output of
        this function serves to input for the parameter `list_fields_to_select`
                                                        of      the      methods      <a
href="#src.pgOperations.pgOperations.PgOperations.pgSelect">pgSelect</a>,
        or <a href="#src.pgOperations.pgOperations.PgOperations.pgUpdate">pgUpdate</a>.

        Examples:
            >>>gf=pg.SelectGeometryFormat()
            >>>gfo=pg.SelectGeometryFieldOptions(geom_field_name='geom',
                select_geometry_format=gf.geojson, epsg_to_reproject='25831')#

>>>fieldNames=pgo.pgGetTableFieldNames('d.points',gfo,list_fields_to_remove=['descriptio
n'],
                returnAsString=True)
            >>>wc=pg.WhereClause(where_clause='gid=%s',where_values_list=[3])
                                                >>>res=pgo.pgSelect(table_name='d.points',
string_fields_to_select=fieldNames,whereClause=wc)

        Args:
            table_name:
            selectGeometryFieldOptions:
            list_fields_to_remove:
            returnAsString:
            print_query:

        Returns:
            If `returnAsString` is `False`, a list of the field names the table.
            If `returnAsString` is `True`, a string with field names of the a table
        """

                  consulta="SELECT  column_name  FROM  information_schema.columns  WHERE
table_schema=%s and table_name = %s";
        lis=table_name.split(".")

        cursor=self.pgConnection.cursor
        cursor.execute(consulta,lis)
        if cursor.rowcount==0:
            return None

        listaValores=cursor.fetchall()#es una lista de tuplas.
                #cada tupla es una fila. En este caso, la fila tiene un
                #unico elemento, que es el nombre del campo.

        if selectGeometryFieldOptions is not None:
            if (selectGeometryFieldOptions.geom_field_name,) not in listaValores:
                  raise Exception("pgGetTableFieldNames. The geometry field name {0} is
not  a  field  of  the  table  {1}".format(selectGeometryFieldOptions.geom_field_name,
```

```
table_name))

        listaNombreCampos=[]
        for fila2 in listaValores:
            valor=fila2[0]
            if list_fields_to_remove is not None:
                if valor in list_fields_to_remove:
                    continue
            if selectGeometryFieldOptions is not None:
                if valor==selectGeometryFieldOptions.geom_field_name:
                    if selectGeometryFieldOptions.epsg_to_reproject is not None:
                                                                    if   'binary'   ==
selectGeometryFieldOptions.select_geometry_format:

valor='st_transform({geom_field_name},{epsg_to_reproject})'.format(
                                    geom_field_name = valor,

epsg_to_reproject=selectGeometryFieldOptions.epsg_to_reproject)
                                                                    elif   'text'   ==
selectGeometryFieldOptions.select_geometry_format:

valor='st_astext(st_transform({geom_field_name},{epsg_to_reproject}))'.format(
                                    geom_field_name = valor,

epsg_to_reproject=selectGeometryFieldOptions.epsg_to_reproject)
                                                                elif   'geojson'   ==
selectGeometryFieldOptions.select_geometry_format:

valor='st_asgeojson(st_transform({geom_field_name},{epsg_to_reproject}))'.format(
                                geom_field_name = valor,

epsg_to_reproject=selectGeometryFieldOptions.epsg_to_reproject)
                    else:
                                                                    if   'binary'   ==
selectGeometryFieldOptions.select_geometry_format:
                                        valor='{geometry_field_name}'.format(geom_field_name =
valor)
                                                                    elif   'text'   ==
selectGeometryFieldOptions.select_geometry_format:
                                    valor='st_astext({geom_field_name})'.format(geom_field_name
= valor)
                                                                elif   'geojson'   ==
selectGeometryFieldOptions.select_geometry_format:

valor='st_asgeojson({geom_field_name})'.format(geom_field_name = valor)
            listaNombreCampos.append(valor)
        self.query=consulta

        if self.global_print_queries or print_query:
            print('pgGetTableFieldNames')
            print('Query: ', consulta)
            print('Fields list:', listaNombreCampos)
```

```
        if returnAsString:
            s=""
            for campo in listaNombreCampos:
                s=s + campo + ","
            return s[:-1]
        else:
            return listaNombreCampos

    def pgTableExists(self, table_name_with_schema: str, print_query: bool=False)->bool:
        """
        Returns True or False, depending on if the table exists in the database or not.
        table_name_with_schema: table name included the schema, e.g. "d.boundary".

        Args:

            table_name_with_schema: The table name, e.g. "d.boundary"
            print_query: For debugging purposes. If true will print the queries.

        Examples:

            >>>res=pgo.pgTableExists(table_name_with_schema='d.points')

        Returns:
            True or False, depending on if the table exists in the database or not.
        """

        l=table_name_with_schema.split(".")
        table_schema=l[0]
        table_name=l[1]
             cons="SELECT  EXISTS  (SELECT  1  FROM     information_schema.tables  WHERE
table_schema = %s AND    table_name = %s)"
        self.pgConnection.cursor.execute(cons,[table_schema, table_name])
        if self.global_print_queries or print_query:
            print('pgTableExists')
            print('Query:', cons)
        r=self.pgConnection.cursor.fetchall()
        return r[0][0]

    def pgCreateTable(self,table_name_with_schema:str, fields_definition:str,
        delete_table_if_exists: bool=False, print_query: bool= False)->bool:
        """
        Creates a table. If the table already exists this method can delete it before.
        Returns `True` if the table has been created, or `False` if the table
        already existed, and the parameter `delete_table_if_exists` was set to `False`.

        See an example of use in
        <a href="../tutorials/#delete-rows-and-files">Delete rows and files</a>.

        Examples:

            >>>r=pgo.pgCreateTable(table_name_with_schema="d.customers",
            >>>    fields_definition="gid serial, name varchar, img varchar",
            >>>    delete_table_if_exists= True,print_query=False)
```

```
        Args:

                    table_name_with_schema: The table name, including the schema, e.g.
'public.customers'.
                fields_definition: String with the fields definitions. If any field name
starts
                 with a number, or has a space, must be double quoted, e.g.

                     >>>fields = 'gid serial primary key, natcode varchar,
                         nameunit varchar,"{fieldName}" varchar'.format(
                         fieldName='25_utm')
               delete_table_if_exists: If True will delete the table before create it. If
False,
                 if the table exists Psycopg2 will raise an error.
             print_query: For debugging purposes. If true the delete and create table
                 sentences will be printed.

        Returns:

             True if the table has been created, or false if the table already existed,
                 and was not deleted and created again, because the argument
                 `delete_table_if_exists` was False.
        """
        schema=table_name_with_schema.split(sep='.')[0]
        tableName=table_name_with_schema.split(sep='.')[1]
                    if  self.pgTableExists(table_name_with_schema=table_name_with_schema,
print_query=print_query):
             if delete_table_if_exists:
                                                                    cons='drop    table
"{schema}"."{tableName}"'.format(schema=schema,tableName=tableName)
                 self.pgConnection.cursor.execute(cons)
                 if self.global_print_queries or print_query:
                     print('pgCreateTable')
                     print("Query to delete the table: ", cons)
             else:
                 return False
        cons='create table "{schema}"."{tableName}" ({fields_definition})'.format(

schema=schema,tableName=tableName,fields_definition=fields_definition)
        #print(cons)
        self.pgConnection.cursor.execute(cons)
        self.pgConnection.conn.commit()

        if self.global_print_queries or print_query:
             print('pgCreateTable')
             print("Query to create the table: ", cons)
        return True

     def pgValueExists(self, table_name_with_schema: str, field_name: str, field_value:
Any, print_query: bool=False)->bool:
        """
        Returns True or False depending whether or not if a value exists in a column.
```

```
        Args:
            table_name_with_schema: Table name included the schema. Ej. "d.linde".
            field_name: Column name, e.g. "username".
            field_value: Any value in the column.
            print_query: For debugging purposes. If True will print
                the query and values in the function.

        Returns:
            True or False, depending on if the value exists.
        """

                cons="SELECT exists (SELECT {0} FROM {1} WHERE {2} = %s LIMIT
1)".format(field_name, table_name_with_schema, field_name)
        self.pgConnection.cursor.execute(cons,[field_value])
        r=self.pgConnection.cursor.fetchall()
        if self.global_print_queries or print_query:
            print('pgValueExists')
            print("Query: ", cons)
            print("Field name: '{0}'. Field value: {1}".format(field_name, field_value))
            print("Exists: ", r[0][0])
        return r[0][0]

         def pgDeleteAllTableRowsFromTableWithColumnValue(self, tableName, columnName,
columnValue):
              #deleteAllTableRowsFromTableWithColumnValue(tableName="public.work_images",
columnName="color_works_gid", columnValue=25)
        #removes all the rows from public.work_images where color_works_gid=25

            return self.pgDelete(table_name=tableName, cond_where=columnName + " =%s",
list_values_cond_where=[columnValue])


# class PgCounters:
#     """
#     Counters are created as sequences, and the sequences are represented
#     in the database as tables. The current value of the counters are get
#     with the sentence `select last_value from sequence_name`.
#     All the counters, or sequences, are stored in the schema `counters`.

#     Counters name should be given without the schema name.
#     The schema `counters` and the table `counters.counters` are automatically
#     created first time a counter is added.
#     The table `counters.counters` contains the counters name and a description.
#      This table is used by the method `getAllCounters`, which lists all the counters
name,
#     description, and current value.

#     You can find examples of use in the
#     <a href="../tutorials/#manage-counters">Manage counters</a> section.
#     """
#     counter_schema='counters'
#     counters_table='counters.counters'
```

```
#     pgo: PgOperations = None
#     def __init__(self, pgOperations: PgOperations) -> None:
#         """
#         Constructor.

#         Examples:

#             >>>counter_name = 'c1'
#             >>>c=pg.PgCounters(pgo)
#             >>>c.addCounter(counter_name,counter_name + ' description')

#         Args:
#             pgOperations: PgOperations instance.
#         """
#         self.pgo=pgOperations

#     def addCounter(self, counter_name:str, counter_description: str, start = 1,
#         incrementBy=1, print_query=False):
#         """
#                   Adds a counter. Creates the schema `counters` and the table
`counters.counters`
#         if they do not exist. Adds to the table `counters.counters` a row with
#         the counter name and the counter description.

#         If the counter already exist, an exception will be raised.

#         If the schema `counters` or the table `counters.counters` already
#         exists this function does not raise any exception.

#         You can find an example of use in the
#         <a href="../tutorials/#add-a-counter">Add a counter</a> section.

#         Args:
#             counter_name: Counter name without schema, e.g. 'visits'.
#             counter_description: Counter description.
#             start: Number to start the counter.
#             incrementBy: Increment to add to the counter.
#             print_query: For debug purposes. In true prints the queries and
#                 values of the query in the function.
#         """
#         if start<1:
#             raise Exception("start can not be less than 1")
#         complete_counter_name=self.counter_schema + '.' + counter_name
#         cons='create schema if not exists counters'
#         self.pgo.pgConnection.cursor.execute(cons)
#                  self.pgo.pgCreateTable(self.counters_table,'gid  serial  primary  key,
counter_name varchar unique, counter_description varchar',False,print_query)

#        # 1. Componer el nombre del esquema y secuencia de forma segura
#              # Usamos Identifier para proteger contra inyección SQL en nombres de
tablas/esquemas
#         complete_counter_name = sql.Identifier(self.counter_schema, counter_name)
```

```
#         # 2. Ejecutar la creación del esquema
#             self.pgo.pgConnection.cursor.execute(sql.SQL("create schema if not exists
counters"))

#         # 3. Componer la query de la secuencia
#         # Usamos sql.Literal para los números, así se inyectan como texto literal
#         # y no como parámetros $1, $2
#          cons = sql.SQL("create sequence {name} as integer start with {start} increment
by {inc}").format(
#             name=complete_counter_name,
#             start=sql.Literal(start),
#             inc=sql.Literal(incrementBy)
#         )


#         if self.pgo.global_print_queries or print_query:
#             print('addCounter')
#             print('Create sequence: ', cons)

#         if self.pgo.autoCommit:
#             self.pgo.pgConnection.commit()
#                                       fav=FieldsAndValues({'counter_name':counter_name,
'counter_description':counter_description})
#         self.pgo.pgInsert(self.counters_table,fav,None,print_query)

#     def deleteCounter(self,counter_name: str, print_query=False)->int:
#         """
#         Deletes a counter,  or sequence. Also deletes de corresponding row
#         in the table `counters.counters`.
#         If the counter does not exist, this method do not raises any exception.

#         You can find an example of use in the
#         <a href="../tutorials/#delete-a-counter">Delete a counter</a> section.
#         Args:
#             counter_name: Counter name without schema, e.g. 'visits'.
#             print_query: For debug purposes. In true prints the queries and
#                 values of the query in the function.

#         Returns:
#                 An integer with the number of rows of the table `counters.counters`
deleted.

#         """
#         complete_counter_name=self.counter_schema + '.' + counter_name
#         cons='drop sequence if exists {0}'.format(complete_counter_name)
#         self.pgo.pgConnection.cursor.execute(cons)
#         if self.pgo.autoCommit:
#             self.pgo.pgConnection.commit()

#         wc=WhereClause('counter_name=%s',[counter_name])
#         n=self.pgo.pgDelete(self.counters_table,wc,print_query)

#         if self.pgo.global_print_queries or print_query:
```

```
#             print('deleteCounter')
#             print('Query', cons)
#             print('Sequences deleted: ', n)

#         return n

#     def incrementCounter(self, counter_name: str, print_query=False) -> int:
#         # 1. Construimos el nombre completo como un string simple
#         full_name = f"{self.counter_schema}.{counter_name}"

#               # 2. Usamos sql.Literal para que psycopg lo envíe como una cadena:
'counters.c1'
#         # Esto es lo que nextval espera recibir.
#         query = sql.SQL("SELECT nextval({name})").format(
#             name=sql.Literal(full_name)
#         )

#         if print_query:
#             # Ahora la query será: SELECT nextval('counters.c1')
#             print("Query:", query.as_string(self.pgo.pgConnection.cursor))

#         try:
#             self.pgo.pgConnection.cursor.execute(query)
#             result = self.pgo.pgConnection.cursor.fetchone()
#             return result[0] if result else None
#         except Exception as e:
#             # Si aquí te dice que la relación no existe,
#             # asegúrate de haber hecho COMMIT tras crearla en addCounter.
#             print(f"Error al incrementar: {e}")
#             raise

#     def getCounterValue(self,counter_name:str, print_query=False)->int:
#         """
#         Returns the current counter value.

#         You can find an example of use in the
#         <a href="../tutorials/#get-the-current-counter-value">
#             Get the current counter value</a> section.</a>

#         Args:

#             counter_name: Counter name without schema, e.g. 'visits'.
#             print_query: For debug purposes. In true prints the queries and
#                 values of the query in the function.

#         Returns:
#             The current counter value.
#         """
#         complete_counter_name=self.counter_schema + '.' + counter_name
#         cons='select last_value from {0}'.format(complete_counter_name)
#         self.pgo.pgConnection.cursor.execute(cons)
#         r=self.pgo.pgConnection.cursor.fetchall()
```

```
#         if self.pgo.global_print_queries or print_query:
#             print('getCounterValue')
#             print('Quer: y', cons)
#             print('Current counter value: ', r[0][0])

#         return r[0][0]
#     def getAllCounters(self, print_query=False)->list:
#         """
#         Returns all the counters name, description and current values in a list of
#         dictionaries.

#         <a href="../tutorials/#get-all-the-counter-name-description-and-values">
#             Get all the counter name, description and values</a> section.</a>

#         Args:

#             print_query: For debug purposes. In true prints the queries and
#                 values of the query in the function.

#         Returns:
#             All the counters name, description and current values in a list of
#         dictionaries.
#         """
#         r=self.pgo.pgSelect(self.counters_table,'*', print_query=print_query)
#         for row in r:
#             currentValue=self.getCounterValue(row['counter_name'])
#             row['value']=currentValue
#         return r
```